MyCodeSchool

Code review — sebastian-docker-compose-all-apps

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 1 — 2026-10-20 — sha 9441261

Scop: `docker-compose-all.yaml`, singurul fișier din repo. Poarta de compilare: nu există cod de build, deci am pornit stack-ul efectiv cu `docker compose up` și am exersat endpoint-urile. Fiecare 🔴 de mai jos a fost reprodus.

🔴 Critice

B1 🔴 Două imagini din compose nu există pe Docker Hub, deci stack-ul nu pornește deloc

`docker-compose-all.yaml:67`

CUM E ACUM: `boats-users` cere `sebi123q/boats-users-app:1.0`, dar repo-ul acela de pe Hub n-are niciun tag (imaginea ta e publicată ca `sebi123q/boat-users-api:1.0`). Același bug la `:83`: `solarsyncbroker` cere `sebi123q/solarsyncbroker-app:2.0`, iar pe Hub există doar `latest`, `sha-ab9dce1`, `sha-b27876a` și `sha-0d47d3d`. Rezultatul: `docker compose up -d` se oprește la pull și nu pornește nici serviciile care sunt corecte (mysql, school, cars, shop).

Rulare: `docker compose -p sebireview -f docker-compose-all.yaml up -d` a dat:

```
Image sebi123q/boats-users-app:1.0 Error
failed to resolve reference
"docker.io/sebi123q/boats-users-app:1.0": not found
Image sebi123q/solarsyncbroker-app:2.0 Error ... not found
Error response from daemon
```

Niciun container pornit. Cu un override `boat-users-api:1.0` și `solarsyncbroker-app:sha-ab9dce1`, ambele pornesc: `Started BoatsUsersApplication` pe 8085 și `Started SolarSyncBrokerApiApplication` pe 8086.

DE CE: `docker compose up` rezolvă mai întâi toate imaginile care lipsesc local și abia după ce le are pe toate creează containerele. Un tag care nu există în registry înseamnă „failed to resolve reference ...

not found", iar compose abandonează toată operația, nu doar serviciul acela. De asta un singur nume greșit blochează tot fișierul.

Pe mașina ta merge probabil pentru că ai imaginile vechi deja în cache (ai făcut build local cu numele acesta), așa că pull-ul nu se mai face. Pe un calculator curat sau pe EC2 nu le are nimeni. Numele și tag-ul din compose trebuie să fie exact cele pe care le-a publicat `docker push` sau CI-ul, iar CI-ul tău pune tag-uri `sha-*`, nu `2.0`.

FIX: `image: sebi123q/boat-users-api:1.0` la `:67` și `image: sebi123q/solarsyncbroker-app:sha-ab9dce1` la `:83`, sau publici efectiv tag-urile pe care le ai acum în compose. Verifici pe un docker fără cache: `docker compose -f docker-compose-all.yaml pull`.

- același bug la:
- `docker-compose-all.yaml:67` — `sebi123q/boats-users-app:1.0` (semnalat). Repo-ul `boats-users-app` e gol pe Hub; există `sebi123q/boat-users-api:1.0`.
- `docker-compose-all.yaml:83` — `sebi123q/solarsyncbroker-app:2.0`. Tag-ul `2.0` nu există; există doar `latest`, `sha-ab9dce1`, `sha-b27876a`, `sha-0d47d3d`.
- Nu mai există alte încălcări. Celelalte 4 imagini (`mysql:8.4` la `:3`, `sebi-school-app:2.0` la `:20`, `cars-app:1.0` la `:35`, `sebi-online-shop-app:1.0` la `:51`) există pe Hub.

B2 🔴 Același JWT_SECRET_KEY în cars, shop și boats (și ca default în school): un token de la o aplicație deschide celelalte aplicații

`docker-compose-all.yaml:44`

CUM E ACUM: cars (`:44`), sebi-online-shop (`:60`) și boats-users (`:76`) primesc exact aceeași cheie. school nu primește deloc `JWT_SECRET_KEY`, deci folosește default-ul din imagine (`application.yml`: `${JWT_SECRET_KEY:asdasd...ABCDEF}`), care e tot cheia aceasta. Un user creat doar în cars citește lista de clienți din shop și intră și în school.

Rulare:

```
POST :8083/api/v1/user/register  -> 201
POST :8083/api/v1/user/login     -> token (iss controllerPractice-api,
                                    sub x@cars.ro)
GET  :8084/api/v2/products/all   fără token -> 401
GET  :8084/api/v2/products/all   token cars -> [] [200]
GET  :8084/api/v2/costumers/all  token cars -> [] [200]
select count(*) from online_shop_db.users    -> 0
GET  :8081/api/v2/students/1     fără token -> 401
GET  :8081/api/v2/students/1     token cars -> 404 "Student does not exist"
```

DE CE: un JWT semnat HS512 e verificat doar cu cheia secretă. Aplicația verifică semnătura și expirarea, apoi crede `sub` și `authorities` din token, fără să caute userul în baza ei. Dacă patru aplicații au aceeași cheie, fiecare poate emite token-uri valide pentru toate celelalte.

Asta ai văzut în rulare: shop are 0 useri în `online_shop_db.users` și totuși a răspuns 200 la token-ul din cars, iar school a trecut de filtru și a răspuns 404 de business, nu 401. Faptul că school nu are variabila setată e capcana „default-ul din cod e tot secretul”: nu vezi cheia în compose, dar ea e acolo.

FIX: o cheie diferită pentru fiecare serviciu, citită din `.env` (`JWT_SECRET_KEY: ${CARS_JWT_SECRET}`), cu `.env` în `.gitignore` și un `.env.example`. Adaugi `JWT_SECRET_KEY` și la school. Ca regulă: în imagine nu se pune default pentru secret.

- același bug la:
- `docker-compose-all.yaml:44` (cars) — cheia identică, semnalată.
- `docker-compose-all.yaml:60` (sebi-online-shop) — aceeași valoare literală, reprodusă prin rulare.
- `docker-compose-all.yaml:76` (boats-users) — aceeași valoare literală (verificat doar static; imaginea originală nu putea fi trasă).
- `docker-compose-all.yaml:24-28` (school) — nu setează `JWT_SECRET_KEY`; default-ul din imagine s-a dovedit aceeași cheie ca la cars. Adaugă cheie proprie în `environment`.
- `docker-compose-all.yaml:92` (solarsyncbroker) — cheie diferită, deci nu încalcă regula cross-app, dar e literală în fișier, comisă în git (placeholder „dev-only...change-in-prod”).
- `docker-compose-all.yaml:44, 60, 76, 92` — toate cele patru chei JWT sunt literale în YAML versionat. Mută-le în `.env` (necomis) și interpolează `${JWT_SECRET_KEY_CARS}` etc., câte una distinctă per aplicație.

- `docker-compose-all.yaml:6, 12, 28, 43, 59, 75, 91` — parola `root` / `root` pentru MySQL, folosită de toate aplicațiile cu user root, cu portul 3307 expus pe host (`:8`). Aceeași clasă de secrete hardcodate, plus fără utilizator DB separat per aplicație.

- Observație în afara regulii: răspunsul `POST /api/v1/user/register` din cars are `lastName` și `email` inversate (`{firstName:Xxxx, lastName:x@cars.ro, email:Yyyy}`). E un bug în codul imaginii `cars-app` , nu în repo.

🟡 Importante

- **M1** `docker-compose-all.yaml:20,35,51,67,83` — imaginile sunt doar `linux/amd64` și n-ai `platform:` . Pe arm64 (Mac M, Graviton) `docker pull` dă `no matching manifest for linux/arm64/v8` . Rezolvi cu build multi-arch (`docker buildx --platform linux/amd64,linux/arm64`) sau cu `platform: linux/amd64` pe fiecare serviciu.
- **M2** `docker-compose-all.yaml:6,27-28,42-43,...` — toate aplicațiile folosesc root/root din MySQL, deci oricare dintre ele poate citi sau șterge bazele celorlalte. Fă un user per bază (script în `/docker-entrypoint-initdb.d`) și pune parolele în `.env` .
- **M3** `docker-compose-all.yaml:8` — MySQL e expus pe host (3307) cu root/root, iar pe EC2 portul acesta ajunge pe internet. Containerele vorbesc între ele prin rețeaua compose, deci scoate `ports` sau leagă-l la `127.0.0.1:3307:3306` .
- **M4** `docker-compose-all.yaml:27-28` — school ignoră `DB_USERNAME` și `DB_PASSWORD` , pentru că imaginea `sebi-school-app:2.0` are în `application.yml` `username: root` scris direct, fără `${...}` . Variabilele din compose nu au niciun efect, iar dacă schimbi parola, school pică.
- **M5** `docker-compose-all.yaml:20` — pe Hub, `sebi-school-app:1.0` e publicat pe 2026-09-02, iar `2.0` pe 2026-07-29, deci tag-ul „mai mare" e imaginea mai veche. Verifică ce cod rulezi și folosește tag-uri `sha-*` din CI.

🟢 Cleanups

- **C1** `docker-compose-all.yaml:3,21,36,52,68,84` — `container_name` fix (mysql, school, cars...) blochează un al doilea stack pe aceeași mașină și intră în conflict cu orice alt container numit `mysql` . Îl poți scoate și lași compose să pună prefixul de proiect.
- **C2** `docker-compose-all.yaml:52,68,84` — numele nu sunt consecvente: serviciul se numește `sebi-online-shop` , iar containerul `sebi-online-shop-app` . Alege o singură convenție.
- **C3** `docker-compose-all.yaml:26,41,...` — `createDatabaseIfNotExist=true` și `useSSL=false` sunt repetate în 5 URL-uri. Merge cu un anchor YAML (`x-db-params`) sau cu `.env` .

Ce e bine: `depends_on: condition: service_healthy` împreună cu healthcheck-ul funcționează (toate cele 4 aplicații se conectează din prima la Hikari), câte o bază per aplicație e creată corect

(`school_db` , `cars_db` , `online_shop_db`), iar `SERVER_PORT` e respectat (Tomcat pe 8081/8083/8084/8085/8086).

Before / After pentru 🔴

	Acum	Corect
B1 `:67`	`image: sebi123q/boats-users-app:1.0`	`image: sebi123q/boat-users-api:1.0`
B1 `:83`	`image: sebi123q/solarsyncbroker-app:2.0`	`image: sebi123q/solarsyncbroker-app:sha-ab9dce1`
B2 `:44`	`JWT_SECRET_KEY: asdasd...ABCDEF` (la fel în cars, shop, boats)	`JWT_SECRET_KEY: ${CARS_JWT_SECRET}` și câte o variabilă distinctă în `.env` pentru fiecare serviciu
B2 `:24-28`	school fără `JWT_SECRET_KEY` (default din imagine)	`JWT_SECRET_KEY: ${SCHOOL_JWT_SECRET}`

Q&A

1. La tine pe calculator `docker compose up` pornește tot, dar pe o mașină curată pică la pull. Ce are mașina ta și n-are cealaltă? Cum verifici înainte de commit că fiecare `image:` din compose chiar există în registry?
2. Shop nu are niciun user, și totuși a acceptat token-ul emis de cars. Ce verifică exact filtrul JWT când primește un token, și ce NU verifică? Ce s-ar întâmpla dacă cineva află cheia din fișierul tău, care e public pe GitHub?
3. Ai pus `DB_USERNAME` și `DB_PASSWORD` și la school, dar aplicația le ignoră. Cum ți-ai fi dat seama din rulare, fără să deschizi jar-ul? Ce schimbi în `application.yml` ca variabila să conteze?